

3-1 · Pandas를 활용한 데이터 처리 · 실습 1H

데이터 불러오기와 저장

3주차 공용 데이터를 만들고, 첫인상 5종 세트

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

① ~/data-eng/week3 에 venv 를 만들고 pandas·pyarrow 를 설치한다

② make_orders.py 로 orders.csv(1015행)를 생성한다

③ shape·dtypes·head·info·describe 로 첫인상을 파악한다

④ Parquet 로 저장해 크기를 비교하고 필요한 열만 읽어 본다

make_orders.py 는 지우지 마세요 — 3-5 에서 원본이 필요할 때 다시 실행합니다

여섯 단계 — ~/data-eng/week3 한 폴더에서

2주차와 다른 폴더에 새 가상환경을 만듭니다 — 주차마다 필요한 라이브러리가 다르기 때문

1. 준비	2. 생성	3. 첫인상	4. dtype	5.6. Parquet
venv pandas·pyarrow	make_orders.py 1015행	shape·dtypes head·info	결측이 타입을 바꾼다	크기 비교 열만 읽기

1단계

폴더 · 가상환경 · 설치 · 확인

폴더	<code>mkdir -p ~/data-eng/week3 · cd ~/data-eng/week3</code>
가상환경	<code>python3 -m venv .venv</code>
활성화	<code>source .venv/bin/activate</code>
설치	<code>pip install pandas pyarrow</code>
확인	<code>python -c "import pandas as pd; print(pd.__version__)"</code>

확인 포인트: 마지막 명령에서 Pandas 버전 번호(예: 2.x.x)가 출력되면 준비 완료 · WSL2·macOS·리눅스 모두 같다

make_orders.py — 시드를 고정한 공용 데이터

1 `rng = np.random.default_rng(42), n = 1000`
누가 몇 번을 실행해도 같은 데이터 — 6개 열, 6/1 부터 60일치 주문

2 **품질 문제를 일부러 심는다**
price 결측 30건 · category '식품' 공백 오염 20건 · 중복 15건

3 **날짜를 문자열로 바꿔 CSV 저장**
`orders.to_csv('orders.csv', index=False, encoding='utf-8')`

4 **실행하고 확인**
`python3 make_orders.py → ls -lh orders.csv · head -n 3 orders.csv`

기대 출력

1015 행
저장 완료

첫인상 5종 세트 — 받자마자 하는 습관

명령	보여 주는 것
df.shape	(행, 열)
df.dtypes	열마다 타입
df.head(3)	앞 3줄
df.info()	결측 수까지 한눈에
df.describe()	숫자 열의 기초 통계

df = pd.read_csv('orders.csv') — 어떤 데이터를 받아도 shape·dtypes·head 부터

확인 포인트: shape (1015, 6) · order_date 가 object — CSV 는 타입을 저장하지 않아 날짜가 문자열로 읽힌 것 · price non-null 985

정수만 넣었는데 float64 — 결측이 타입을 바꾼다

price — float64

결측 30건 — NaN 을 담을 수 있는 타입이 float 뿐

- `df['price'].dtype → float64`
- `df['price'].isna().sum() → 30`
- `head(3).tolist()` 에 1000.0 으로 보인다

VS

quantity — int64

결측 0건 — 정수 그대로

- `df['quantity'].dtype → int64`
- `df['quantity'].isna().sum() → 0`
- 버그가 아니라 NaN 을 위한 정상 동작

CP949 로 저장한 CSV 를 그냥 읽으면

한국 공공데이터는 CP949 가 많습니다 — 오류를 한 번 보고 나면 해결이 남습니다

1. CP949 저장

```
to_csv(...,  
encoding='cp949')
```

2. 그냥 읽기

```
pd.read_csv  
('orders.csv')
```

3. 오류 확인

```
Unicode  
DecodeError
```

4. 고쳐 읽기

```
encoding=  
'cp949'
```

5. 원본 복구

```
make_orders.py  
다시 실행
```

Parquet 저장 · 크기 비교 · 열만 읽기

1 `df.to_parquet('orders_raw.parquet', index=False)`
읽어 둔 df 를 그대로 Parquet 로 저장

2 `os.path.getsize` 로 두 파일 크기 비교
Parquet 가 CSV 의 절반 이하 — 지금은 1,015행이라 차이가 크지 않다

3 `pd.read_parquet(..., columns=['category', 'price'])`
디스크에서 그 두 열만 읽는다 → (1015, 2)

4 `pd.read_csv(..., usecols=['category', 'price'])`
결과는 같은 (1015, 2) 지만 파일 전체를 읽은 뒤 버린다

3-3 에서

**100만 행으로
다시 켜다**

이 1시간이 끝날 때 확인할 것

- 1 `pd.__version__` 이 2.x.x 로 찍힌다
- 2 '1015 행 저장 완료' · `orders.csv` 가 60KB 안팎이다
- 3 `shape` 가 (1015, 6) 이고 `order_date` 가 object 다
- 4 `price` 는 float64(결측 30건), `quantity` 는 int64 다
- 5 `orders_raw.parquet` 가 CSV 의 절반 이하다
- 6 `columns=['category', 'price']` 로 읽어 (1015, 2) 가 나온다

3-1 · 실습 1H 마무리

다음은 선택·필터·변환 실습 (1.5H)

행과 열을 골라내고, 잘못된 타입을 바로잡고, 파생 컬럼을 만들어 3-2 가 쓸 orders_step1.parquet 를 만듭니다

✓ loc·iloc·불리언 인덱싱으로 고르기

✓ str.strip() 을 빼면 20건이 새는 것 확인

✓ to_datetime·assign·.dt 로 변환

✓ 결측 처리 세 방법과 Parquet 저장

가져갈 것

orders.csv · venv 가 켜진 터미널